Example: Find the value of the following 32 bit binary string representing a single precision IEEE floating point format: 0 10000010 11010000 00000000 0000000. The value is calculated by parsing the number into different fields, namely sign bit, exponent and mantissa, and then computing the value of each field to get the final value, as shown in Table 3.4.

Table 3.4 Value calculated by parsing and then computing the value of each field to get the final value, $+14.5_{10}$ (see text)

Sign bit	Exponent		Mantissa
0	10000010		11010000 00000000 0000000
$(-1)^0$	$\times\ 2^{(130-127)}$	$\times$	$(1\ .\ 1\ 1\ 0\ 1)_2$
1	$\times\ 2^{(3)}$	$\times$	$(1 + 0.5 + .25 + 0 + 0.0625)_{10}$
$(+1)$	$\times\ 2^{(3)}$	$\times$	$(1.8125)_{10}$

Floating-point Format

- Two Representations in IEEE 754
  - Single Precision
  - Double Precision

$$x = (-1)^s \times 1 \times m \times 2^{e\ \ b}$$

- Where
  - s = sign
  - m = mantissa
  - e = excess exponent

Example: This example represents -12.25 in single precision IEEE floating point format. The number -12.25 in sign magnitude binary is -00001100.01. Now moving the decimal point to bring it into the right format: $-1100.01 \times 2^0 = -1.10001 \times 2^3$. Thus the normalized number is -1.10001×2^3.

Sign bit (s) = 1
Mantissa field(m) = 10001000_00000000_0000000
Exponent field(e) = 3 + 127 = 130 = 82 h = 1000 0010

So the complete 32 bit floating point number in binary representation is:

1_10000010_10001000_00000000_0000000

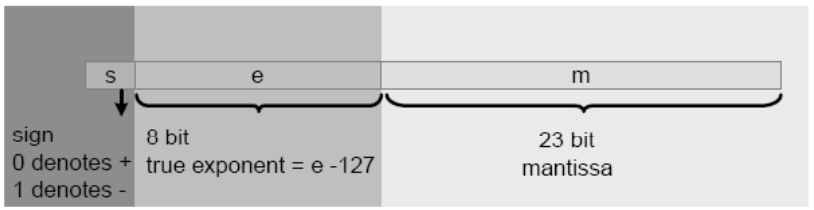

Figure 3.2 IEEE format for single precision 32 bit floating point number

3.4.3 *Floating-point Multiplication*

The following steps are used to perform floating point multiplication of two numbers.

S0 Add the two exponents e_1 and e_2. As inherent bias in the two exponents is added twice, subtract the bias once from the sum to get the resultant exponent e in correct format.
S1 Place the implied 1 if the operands are saved as normalized numbers. Multiply the mantissas as unsigned numbers to get the product, and XOR the two sign bits to get the sign of the product.
S2 Normalize the product if required. This puts the result back to IEEE format. Check whether the result underflows or overflows.
S3 Round or truncate the mantissa to the width prescribed by the standard.

Example: Multiply the two numbers below stored as 10 bit floating point numbers, where 4 bits and 5 bits, respectively, are allocated for the exponent and mantissa, 1 bit is kept for the sign and 7 is used as bias:

$$3.5 \rightarrow 0\_1000\_11000$$
$$5.0 \rightarrow 0\_1001\_01000$$

S0 Add the two exponents and subtract the bias 7 (=0111) from the sum:

$$1000 + 1001 - 0111 = 1010$$

S1 Append the implied 1 and multiply the mantissa using unsigned $\times$ unsigned multiplication:

$$\begin{array}{r} 1.11 \\ 1.01 \\ \hline 111 \end{array}$$

$$\begin{array}{r} 00\text{x} \\ 111\text{xx} \\ \hline 10.0011 \end{array}$$

S2 Normalize the result:

$$(10.0011)_b \times 2^3 \rightarrow (1.00011)_b \times 2^4$$
$$(1.00011)_b \times 2^4 \rightarrow (17.5)_{10}$$

S3 Put the result back into the format of the operands. This may require dropping of bits but in this case no truncation is required and the result is $(1.00011)_b \times 2^4$ stored as 0 1011 00011.

3.4.2 *Floating-point Arithmetic Addition*

The following steps are used to perform floating point addition of two numbers:

S0 Append the implied bit of the mantissa. This bit is 1 or 0 for normalized and denormalized numbers, respectively.
S1 Shift the mantissas from S0 with smaller exponent e_s to the right by $e_l - e_s$, where e_l is the larger of the two exponents. This shift may be accomplished by providing a few guard bits to the right for better precision.
S2 If any of the operands is negative, take two's complement of the mantissa from S1 and then add the two mantissas. If the result is negative, again takes two's complement of the result.
S3 Normalize the sum back to IEEE format by adjusting the mantissa and appropriately changing the value of the exponent e_l. Also check for underflow and overflow of the result.
S4 Round or truncate the resultant mantissa to the number of bits prescribed in the standard.

Example: Add the two floating point numbers below in 10 bit precision, where 4 bits are kept for the exponent, 5 bits for the mantissa and 1 bit for the sig. Assume the bias value is 7:

$$0\_1010\_00101$$
$$0\_1001\_00101$$

Taking the bias $+7$ off from the exponents and appending the implied 1, the numbers are:

$$1.00101_b \times 2^3 \text{ and } 1.00101_b \times 2^2$$

S0 As both the numbers are positive, there is no need to take two's complements. Add one extra bit as guard bit to the right, and align the two exponents by shifting the mantissa of the number with the smaller exponent accordingly:

$$1.00101_b \times 2^3 \rightarrow 1.001010_b \times 2^3$$
$$1.00101_b \times 2^2 \rightarrow 0.100101_b \times 2^3$$

S1 Add the mantissas:

$$1.001010_b + 0.100101_b = 1.101111_b$$

S2 Drop the guard bit:

$$1.10111_b \times 2^3$$

S3 The final answer is $1.10111_b \times 2^3$, which in 10 bit format of the operands is 0 1010 10111.